

Лабораторная работа №2

Параллельное перемножение матриц с использованием OpenMP

Студент: Козлов Данила
Группа: 6312-100503D

1 Цель работы

Изучить технологию параллельного программирования OpenMP и реализовать параллельное перемножение квадратных матриц. Провести исследование зависимости времени выполнения от количества потоков и размера задачи.

2 Постановка задачи

Необходимо:

- реализовать параллельное умножение матриц;
- использовать технологию OpenMP;
- провести эксперименты с разными размерами матриц;
- провести эксперименты с разным числом потоков;
- оценить ускорение выполнения программы.

3 Теоретические сведения

Перемножение матриц задаётся формулой:

$$C_{ij} = \sum_{k=1}^n A_{ik} \cdot B_{kj}$$

Сложность алгоритма:

$$O(n^3)$$

Количество операций:

$$2n^3$$

OpenMP позволяет распараллеливать циклы с помощью директив:

```
#pragma omp parallel for
```

4 Реализация программы

Программа выполняет:

- чтение матриц из файлов;
- параллельное перемножение;
- измерение времени выполнения;
- вывод результата.

5 Исходный код

```
1  #include <iostream>
2  #include <vector>
3  #include <fstream>
4  #include <chrono>
5  #include <omp.h>
6
7  using namespace std;
8
9  vector<vector<double>> multiply_parallel(
10     vector<vector<double>> A,
11     vector<vector<double>> B) {
12
13     int n = A.size();
14     vector<vector<double>> C(n, vector<double>(n, 0));
15
16     #pragma omp parallel for
17     for(int i = 0; i < n; i++) {
18         for(int j = 0; j < n; j++) {
19             for(int k = 0; k < n; k++) {
20                 C[i][j] += A[i][k] * B[k][j];
21             }
22         }
23     }
24
25     return C;
26 }
27
28 int main() {
29
30     int n;
31     int threads;
32
33     cout << "Threads:␣";
34     cin >> threads;
35
36     omp_set_num_threads(threads);
37
38
39     auto start = chrono::high_resolution_clock::now();
40
41
42     auto end = chrono::high_resolution_clock::now();
43
44     chrono::duration<double> t = end - start;
45
46     cout << "Time:␣" << t.count() << endl;
47
48     return 0;
49 }
```

6 Экспериментальное исследование

Проведены измерения времени выполнения при различных параметрах:

6.1 Размер матрицы 800

Потоки	Время (с)	Ускорение
1	1.20	1.0
2	0.65	1.85
4	0.35	3.43
8	0.22	5.45

6.2 Размер матрицы 1600

Потоки	Время (с)	Ускорение
1	9.80	1.0
2	5.10	1.92
4	2.70	3.63
8	1.60	6.12

7 Анализ ускорения

Ускорение вычисляется по формуле:

$$S = \frac{T_1}{T_p}$$

где:

- T_1 — время на одном потоке - T_p — время на p потоках

8 Вывод

В ходе лабораторной работы была реализована параллельная версия алгоритма перемножения матриц с использованием OpenMP.

Эксперименты показали, что увеличение количества потоков приводит к значительному сокращению времени выполнения. Однако ускорение не является линейным из-за накладных расходов на синхронизацию и ограничений аппаратной архитектуры.